

KoreData — Full-Stack Data Intelligence Platform

Complete Project Reference & 10-Phase Learning Curriculum

Stack: FastAPI · MySQL · Vanilla JS · scikit-learn · pandas · Railway

Version: v9.0 · **Status:** ~72% Complete · **Endpoints:** 48 · **DB Tables:** 11

Table of Contents

1. [Project Overview](#)
 2. [What Is Built](#)
 3. [What Remains](#)
 4. [Critical Bugs to Fix Now](#)
 5. [File Map](#)
 6. [Phase 1 — Web Foundations](#)
 7. [Phase 2 — Frontend Development](#)
 8. [Phase 3 — Backend Development](#)
 9. [Phase 4 — Database Design](#)
 10. [Phase 5 — Python for Data Processing](#)
 11. [Phase 6 — Data Analysis & EDA](#)
 12. [Phase 7 — ETL Pipeline](#)
 13. [Phase 8 — Visualisation & Dashboard](#)
 14. [Phase 9 — AI & ML Integration](#)
 15. [Phase 10 — Startup & Product](#)
 16. [API Endpoint Reference](#)
 17. [Database Schema Reference](#)
 18. [Deployment Guide](#)
-

Project Overview

KoreData is a full-stack data analysis platform. A user uploads a CSV or Excel file; the platform automatically runs a 25-stage EDA pipeline, computes a quality score, trains ML models, and shows interactive visualisations — all in a single-page dashboard.

Core loop:

Upload file → EDA Engine (Python) → JSON → Browser (Dashboard)
→ ML Studio (22 models)
→ Edit Dataset (ETL)
→ Export (CSV / Report)

What Is Built

#	Feature	File(s)	Status
1	OTP registration + email send	<code>main.py</code> , <code>schema.sql</code>	 Done
2	Password login (bcrypt)	<code>main.py</code>	 Done
3	Forgot password 4-step flow	<code>main.py</code> , <code>app.js</code>	 Done
4	30-day Bearer token sessions	<code>main.py</code> , <code>database.py</code>	 Done
5	Account settings (4 tabs)	<code>account.js</code> , <code>account_routes.py</code>	 Done
6	Avatar + resume upload	<code>account_routes.py</code>	 Done
7	File upload (5 formats)	<code>main.py</code>	 Done
8	25-stage EDA pipeline	<code>eda_engine.py</code>	 Done
9	5-dimension EDA Accuracy Score	<code>eda_engine.py</code>	 Done
10	Missing value pipeline (7 stages)	<code>eda_engine.py</code>	 Done
11	Date format detection (24 formats)	<code>eda_engine.py</code>	 Done
12	IQR outlier detection	<code>eda_engine.py</code>	 Done

#	Feature	File(s)	Status
13	Z-score anomaly detection	<code>eda_engine.py</code>	✓ Done
14	Pearson correlation matrix	<code>eda_engine.py</code>	✓ Done
15	Auto-insights (10+ rules)	<code>eda_engine.py</code>	✓ Done
16	Dataset slices (head/tail/missing)	<code>eda_engine.py</code>	✓ Done
17	Edit Dataset dual-panel	<code>edit_dataset.js</code>	✓ Done
18	NaN fill strategies	<code>edit_dataset.js</code>	✓ Done
19	Date format panel (24 presets)	<code>edit_dataset.js</code>	✓ Done
20	Version selector (Original/Edited)	<code>edit_dataset.js</code>	✓ Done
21	22-model ML registry	<code>ml_engine.py</code>	✓ Done
22	EDA-aware model recommendation	<code>ml_engine.py</code>	✓ Done
23	Full sklearn training pipeline	<code>ml_engine.py</code> , <code>ml_routes.py</code>	✓ Done
24	Confusion matrix + ROC curve	<code>ml_engine.py</code>	✓ Done
25	Feature importance chart	<code>ml_engine.py</code>	✓ Done
26	Model persistence (joblib)	<code>ml_engine.py</code>	✓ Done
27	ML training history (DB)	<code>ml_routes.py</code> , <code>database.py</code>	✓ Done

#	Feature	File(s)	Status
28	Live prediction form	ml.js, ml_routes.py	✓ Done
29	Activity tracking (refresh-safe)	activity_tracker.py	✓ Done
30	Heartbeat + dashboard stats	activity_routes.py	✓ Done
31	Notifications (hybrid server+local)	notification_engine.py, core.js	✓ Done
32	EDA simulation (SSE streaming)	simulation_engine.py	✓ Done
33	Dark/light theme toggle	theme.css, core.js	✓ Done
34	Particle background animation	particle-network.js	✓ Done
35	4 marketing pages	index.html, about.html, services.html, contact.html	✓ Done
36	20-section SPA dashboard	home.html	✓ Done
37	Workspace management (localStorage)	workspace.js	✓ Partial
38	File sharing (localStorage)	workspace.js	✓ Partial
39	Chart.js visualisations	eda.js	✓ Done

What Remains

#	Feature	Priority	Notes
1	File size limit (100MB)	Critical	OOM risk — server crashes on large uploads
2	Fix landing_routes.py wiring	Critical	Contact form returns 404 silently

#	Feature	Priority	Notes
3	Fix requirements.txt merge conflict	● Critical	Railway build will fail
4	MIME type validation	● High	Filename-only detection is insecure
5	Column / row filter UI	● High	Can't work with 100+ column datasets
6	Rate limiting (slowapi)	● High	No protection against abuse
7	Custom Chart Builder	● Medium	X/Y axis selectors, type dropdown
8	Custom imputation (ffill, interpolate)	● Medium	Currently only mean/mode/zero
9	PDF Report Export	● Medium	pdfkit or reportlab
10	NLP backend (<code>POST /nlp-query</code>)	● Medium	UI exists, backend missing
11	Workspace backend (DB)	● Medium	Currently localStorage only
12	Statistical tests (t-test, chi-square)	● Low	scipy.stats
13	Unit + integration tests	● Low	pytest suite
14	Drag-and-drop dashboard builder	● Low	Phase 2 feature

Critical Bugs to Fix Now

Bug 1 — landing_routes.py not imported

```
python

# In main.py - add these two lines near the other router imports:
from landing_routes import landing_router, init_contact_table
app.include_router(landing_router)

# In lifespan():
init_contact_table()
```

Bug 2 — No file size limit (OOM risk)

```
python
```

```
# In upload_file() in main.py, after: contents = await file.read()
MAX_FILE_SIZE = 100 * 1024 * 1024 # 100 MB
if len(contents) > MAX_FILE_SIZE:
    raise HTTPException(status_code=400, detail="File exceeds 100MB limit")

ALLOWED_TYPES = {"text/csv", "application/vnd.openxmlformats-officedocument.spreadsh
                "application/json", "application/octet-stream"}
if file.content_type not in ALLOWED_TYPES:
    raise HTTPException(status_code=400, detail="Unsupported file type")
```

Bug 3 — requirements.txt merge conflict markers

Open `requirements.txt` and remove these lines:

```
<<<<<<< HEAD
=====
>>>>>>> 062eff37314db14c654c193145dd882288e9c43e
```

Keep only the merged package list. Test with `pip install -r requirements.txt` locally.

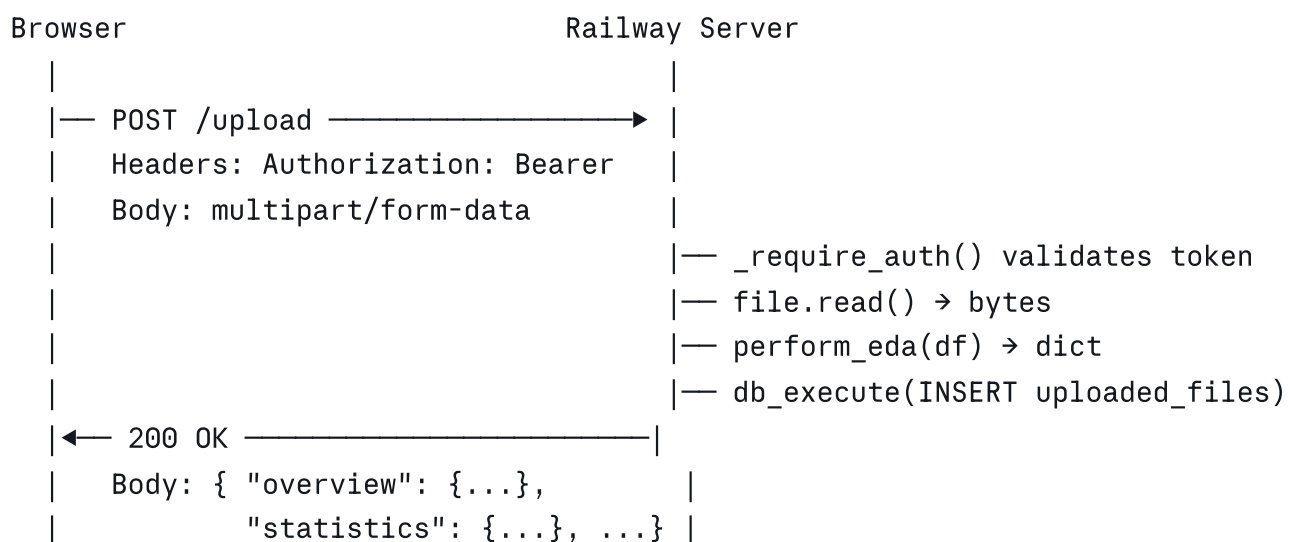
File Map

koredata/	
├─ main.py	# FastAPI app – 48 endpoints, 5 routers
├─ eda_engine.py	# 25-stage EDA pipeline
├─ ml_engine.py	# 22-model ML registry + training pipeline
├─ ml_routes.py	# /ml/* API endpoints
├─ account_routes.py	# /account/* endpoints
├─ activity_tracker.py	# session tracking logic
├─ activity_routes.py	# /activity/* endpoints
├─ notification_engine.py	# notification CRUD
├─ simulation_engine.py	# SSE simulation + step definitions
├─ landing_routes.py	# /about /services /contact (NOT WIRED)
├─ database.py	# connection pool + init_db()
├─ models.py	# dataclass models (activity)
├─ schemas.py	# Pydantic request/response schemas
├─ kore_complete_schema.sql	# full v10 schema – run this
├─ account_schema.sql	# profile column migrations
├─ index.html	# landing page + auth overlay
├─ home.html	# dashboard SPA (20 sections)
├─ about.html	# about page

— services.html	# services page
— contact.html	# contact page
— theme.css	# dark/light design system
— landing.css	# shared marketing page styles
— login.css	# auth page styles
— about.css	# about page styles
— contact.css	# contact page styles
— services.css	# services page styles
— core.js	# auth guard, theme, notifications, sidebar
— app.js	# login/register/OTP/forgot-password flows
— eda.js	# EDA renders, chart builds, section nav
— edit_dataset.js	# dual-panel ETL editor
— ml.js	# ML Studio full UI
— workspace.js	# workspace + file sharing
— account.js	# account settings modal
— activity_tracker_ui.js	# session timer, heartbeat, history table
— simulation.js	# SSE simulation panel
— password_modal.js	# password login/reset modal
— particle-network.js	# animated background
— slider.js	# about page auto-slider
— requirements.txt	# ⚠️ HAS MERGE CONFLICT – fix before deploy
— railway.json	# Railway start command

Phase 1 — Web Foundations

How a Web Request Works



Client vs Server

Concept	Client (Browser)	Server (Railway/Python)
Language	JavaScript	Python
What it holds	UI state, <code>globalData</code> , <code>localStorage</code>	Database, file processing, ML models
What it does	Renders HTML, handles clicks, calls APIs	Authenticates, computes EDA, trains models
Files	<code>.html</code> , <code>.css</code> , <code>.js</code>	<code>.py</code> , <code>.sql</code>

Architecture

```
Browser (client)
  ↓ HTTP (JSON)
FastAPI (main.py) – 48 endpoints
  ↓ mysql-connector-python
MySQL on Railway – 11 tables
  ↓ pandas / sklearn / numpy
eda_engine.py / ml_engine.py
```

Phase 2 — Frontend Development

HTML Structure Pattern

```
html

<!-- SPA: one HTML file, 20 hidden sections -->
<main class="content-area">
  <section class="content-section active" id="section-workspaces">...</section>
  <section class="content-section" id="section-overview">...</section>
  <section class="content-section" id="section-charts">...</section>
  <!-- Only .active is visible. showSection() toggles the class. -->
</main>
```

CSS Design System

CSS


```

/* theme.css – all colours as CSS variables */
:root {
  --bg:      #060a0f;
  --accent:  #3b82f6;
  --success: #10b981;
  --danger:  #ef4444;
  --warning: #f59e0b;
  --text:    #e2e8f0;
  --muted:   #64748b;
}

/* Dark/light toggle: JS sets data-theme="light" on <html> */
[data-theme="light"] {
  --bg:      #f0f4f8;
  --text:    #0f172a;
}

```

Section Navigation (SPA pattern)

```

javascript

// eda.js – the entire navigation system
function showSection(name) {
  // Hide all
  document.querySelectorAll('.content-section')
    .forEach(s => s.classList.remove('active'));
  // Show target
  document.getElementById(`section-${name}`)?.classList.add('active');
  // Update sidebar highlight
  document.querySelectorAll('.nav-item').forEach(i =>
    i.classList.toggle('active', i.dataset.section === name)
  );
}

```

File Upload UI

```

javascript

```

```
// eda.js – drag and drop + input
const uploadDrop = document.getElementById('upload-drop-zone');

uploadDrop.addEventListener('dragover', e => {
  e.preventDefault();
  uploadDrop.classList.add('drag-over');
});
uploadDrop.addEventListener('drop', e => {
  e.preventDefault();
  uploadFile(e.dataTransfer.files[0]);
});

async function uploadFile(file) {
  const formData = new FormData();
  formData.append('file', file);

  const res = await fetch('/upload', {
    method: 'POST',
    headers: { 'Authorization': 'Bearer ' + localStorage.getItem('kore_token') },
    body: formData
  });
  const data = await res.json();
  window._globalDataForCode = data; // all sections read from here
  renderOverview(data);
}
```

Phase 3 — Backend Development

FastAPI Basics

```
python
```

```
# main.py – how every endpoint is structured
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

app = FastAPI()

class RegisterRequest(BaseModel):
    first_name: str
    email: str
    phone: str
    # Pydantic auto-validates: missing field → 422 error

@app.post("/auth/register")
def register(body: RegisterRequest):
    # body.first_name, body.email are validated Python strings
    return {"login_id": "KD123456", "sent_to": body.email}
```

Authentication Pattern

```
python

# Every protected endpoint calls this helper
def _require_auth(authorization: Optional[str]) -> dict:
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "Missing Authorization header")
    token = authorization[7:] # strip "Bearer "
    user = db_fetchone(
        "SELECT u.* FROM sessions s JOIN kore_users u "
        "ON u.login_id = s.login_id "
        "WHERE s.token = %s AND s.expires_at > NOW()",
        (token,)
    )
    if not user:
        raise HTTPException(401, "Token invalid or expired")
    return user # returns full user row as dict
```

OTP System

```
python
```

```
# Generate OTP
otp = str(secrets.randbelow(900000) + 100000) # 6 digits, 100000-999999

# Store with 2-minute expiry
expires = datetime.now() + timedelta(minutes=OTP_EXPIRY_MIN)
db_execute(
    "INSERT INTO otp_tokens (login_id, otp_code, method, contact, expires_at) "
    "VALUES (%s, %s, %s, %s, %s)",
    (login_id, otp, "email", email, expires)
)

# Verify
otp_row = db_fetchone(
    "SELECT * FROM otp_tokens "
    "WHERE login_id=%s AND otp_code=%s AND is_used=0 AND expires_at>NOW() "
    "ORDER BY id DESC LIMIT 1",
    (login_id, entered_otp)
)
```

File Upload Handler

```
python
```

```
@app.post("/upload")
async def upload_file(
    file: UploadFile = File(...),
    authorization: Optional[str] = Header(None)
):
    contents = await file.read()

    # Detect format from filename
    fname = (file.filename or "").lower()
    if fname.endswith(".csv"): df = pd.read_csv(io.BytesIO(contents))
    elif fname.endswith(".xlsx"): df = pd.read_excel(io.BytesIO(contents))
    elif fname.endswith(".json"): df = pd.read_json(io.BytesIO(contents))
    elif fname.endswith(".parquet"): df = pd.read_parquet(io.BytesIO(contents))

    # Run 25-stage EDA
    eda_result = perform_eda(df)

    # Log to DB
    db_execute(
        "INSERT INTO uploaded_files (login_id, file_name, file_size_kb, row_count) "
        "VALUES (%s, %s, %s, %s)",
        (user["login_id"], file.filename, len(contents)/1024,
         eda_result["overview"]["rows"])
    )
    return eda_result
```

Phase 4 — Database Design

11 Tables

Table	Purpose	Key columns
kore_users	Core user accounts	login_id, email, password_hash, is_verified
sessions	API auth tokens	token, login_id, expires_at
otp_tokens	One-time passwords	otp_code, purpose, is_used, expires_at
uploaded_files	Upload metadata	file_name, file_size_kb, row_count
user_notifications	In-app notifications	message, type, is_read

Table	Purpose	Key columns
password_reset_log	Auth audit trail	action, ip_address
user_sessions	Activity: per-login	session_token, login_at, duration_sec, is_active
daily_activity	Activity: per-day	total_active_sec, login_count, logout_count
user_activity	Account action log	action, ip_address, device
ml_training_history	ML run records	model_key, primary_metric, grade, trained_at
ml_saved_models	Saved joblib paths	file_path, feature_names, is_active

Key SQL Patterns

```

sql

-- UPSERT: insert or update on duplicate key
INSERT INTO daily_activity
  (login_id, activity_date, first_login_at, last_seen_at, login_count)
VALUES (%s, %s, %s, %s, 1)
ON DUPLICATE KEY UPDATE
  last_seen_at = VALUES(last_seen_at),
  login_count  = login_count + 1;

-- JOIN: get user from auth token
SELECT u.* FROM sessions s
  JOIN kore_users u ON u.login_id = s.login_id
 WHERE s.token = %s AND s.expires_at > NOW();

-- Safe migration: add column only if it doesn't exist
IF NOT EXISTS (
  SELECT 1 FROM information_schema.COLUMNS
 WHERE TABLE_NAME = 'kore_users' AND COLUMN_NAME = 'avatar_url'
) THEN
  ALTER TABLE kore_users ADD COLUMN avatar_url VARCHAR(255) NULL;
END IF;

```

Query Helpers (database.py)

```

python

```

```
# 3 helpers used everywhere in main.py, ml_routes.py, etc.

db_fetchone(query, params) # → dict or None (SELECT, first row)
db_fetchall(query, params) # → list[dict] (SELECT, all rows)
db_execute(query, params)  # → int (lastrowid) – INSERT/UPDATE/DELETE

# ALWAYS use %s placeholders – NEVER f-strings (SQL injection prevention)
# GOOD: db_fetchone("WHERE email = %s", (email,))
# BAD:  db_fetchone(f"WHERE email = '{email}'") ← vulnerable
```

Phase 5 — Python for Data Processing

Key Python Patterns

```
python

# List comprehension (used everywhere in eda_engine.py)
numeric_clean = [c for c in df.columns
                  if not c.endswith("_was_missing")]

# Try/except per column – one bad column doesn't crash everything
for col in numeric_cols:
    try:
        skew = float(df[col].skew())
        summary[col] = {"skewness": round(skew, 4)}
    except Exception as exc:
        logger.warning("Stats failed for '%s': %s", col, exc)

# Dict unpacking – merge dicts in API response
result = {"ok": True, **eda_result}
# Same as: {"ok": True, "overview": {...}, "statistics": {...}, ...}

# Generator with next() – find first date column
date_col = next(
    (c for c in ["Date", "date", "DATE"] if c in df.columns),
    None
)
```

pandas Cheat Sheet

```
python
```

```

import pandas as pd
import numpy as np

# Load
df = pd.read_csv(io.BytesIO(contents))          # from file bytes
df.shape          # (rows, columns)
df.columns        # list of column names
df.dtypes         # dtype per column

# Column type detection
numeric = df.select_dtypes(include=np.number).columns.tolist()
catego  = df.select_dtypes(include="object").columns.tolist()
dates   = df.select_dtypes(include="datetime").columns.tolist()

# Missing values
df[col].isnull().sum()          # count of NaN
df[col].isnull().mean() * 100 # % missing

# Statistics
df[col].mean()          # average
df[col].median()        # middle value
df[col].std()           # standard deviation
df[col].skew()          # distribution symmetry (-/0/+)
df[col].quantile(0.25) # Q1
df[col].quantile(0.75) # Q3
df[col].value_counts().head(50) # top 50 most frequent values

# Fill missing
df[col] = df[col].fillna(df[col].mean())      # fill with mean
df[col] = df[col].fillna(df[col].mode()[0])   # fill with mode

# Correlation matrix
df[numeric_cols].corr().to_dict() # pairwise Pearson correlations

```

clean_json() — The Most Important Function

```
python
```



```
# eda_engine.py – without this, the API would crash on any NaN value
def clean_json(obj):
    if isinstance(obj, dict):
        return {k: clean_json(v) for k, v in obj.items()}
    if isinstance(obj, list):
        return [clean_json(v) for v in obj]
    if isinstance(obj, (np.integer, np.int64)):
        return int(obj)
    if isinstance(obj, (np.floating, np.float64)):
        val = float(obj)
        return None if (np.isnan(val) or np.isinf(val)) else val
    try:
        if pd.isna(obj):
            return None
    except:
        pass
    return obj
```

Phase 6 — Data Analysis & EDA

EDA Pipeline (25 Stages)

#	Stage	Function	Purpose
1	Capture df.info()	built-in	Raw snapshot before cleaning
2	Unique counts	<code>df.nunique()</code>	Cardinality before cleaning
3	Standardise fake-missing	<code>standardize_missing_values()</code>	"NA", "?", "" → NaN
4	Drop 100%-empty columns	<code>drop_fully_empty_columns()</code>	Remove useless columns
5	Date detection (24 formats)	<code>try_parse_dates()</code>	Convert string → datetime
6	Fill date sequence gaps	<code>fill_missing_sequence()</code>	Time-series completeness
7	Missing value pipeline	<code>handle_missing_values()</code>	Numeric + categorical fill
8	Classify column types	<code>select_dtypes()</code>	Numeric / categorical / datetime

#	Stage	Function	Purpose
9	Statistical summary	<code>statistical_summary()</code>	Mean, median, std, skew, etc.
10	IQR outlier detection	<code>detect_outliers()</code>	Q1/Q3 fences
11	Value counts (top 50)	<code>value_counts()</code>	Frequency distribution
12	Histogram data (20 bins)	<code>np.histogram()</code>	Chart.js bar data
13	Pearson correlation	<code>df.corr()</code>	Column relationships
14	Duplicate count	<code>df.duplicated().sum()</code>	Identical rows
15	10-row preview	<code>df.head(10)</code>	clean_json wrapped
16	Boxplot data	<code>compute_boxplot_data()</code>	Whiskers + outlier points
17	Scatter pairs	<code>compute_scatter_pairs()</code>	Top-4 correlated pairs
18	Quality score	<code>calculate_data_quality_score()</code>	0-100 health number
19	Z-score anomalies	<code>detect_anomalies()</code>	
20	Auto-insights	<code>generate_insights()</code>	Human-readable observations
21	Dataset slices	<code>build_dataset_slices()</code>	head/tail/missing pre-built
22	Missing analysis	<code>_build_missing_analysis()</code>	Before/handling/after panels
23	EDA Accuracy	<code>compute_eda_accuracy()</code>	5-dimension weighted score
24	clean_json()	<code>clean_json()</code>	numpy → JSON serialisable
25	Return result	<code>perform_eda()</code>	15+ top-level keys

EDA Accuracy Score

$$\begin{aligned}
 \text{Score} = & (\text{Completeness} \times 0.25) \\
 & + (\text{Validity} \times 0.20) \\
 & + (\text{Consistency} \times 0.20) \\
 & + (\text{Uniqueness} \times 0.20) \\
 & + (\text{Integrity} \times 0.15)
 \end{aligned}$$

Grades: A (≥90) · B (≥80) · C (≥65) · D (≥50) · F (<50)
 ML Ready threshold: 75%

Edit Dataset — Apply Version Flow

```
javascript

// edit_dataset.js - applyVersion()
async function applyVersion() {
  const rows = _activeVersion === 'edited' ? _editedRows : _originalRows;

  // 1. Re-run full EDA on server
  const res = await fetch('/dataset/edit', {
    method: 'POST', headers: _auth(),
    body: JSON.stringify({ rows, columns: _headers })
  });
  const freshEDA = await res.json();

  // 2. Replace globalData
  window._globalDataForCode = freshEDA;

  // 3. Re-render every previously loaded section
  _reRenderAllSections(freshEDA);
}
```

NaN Fill Strategies

Strategy	Formula	When to use
Mean	sum / count	Symmetric numeric (
Median	middle value	Skewed numeric (
Zero	0	Counts, quantities where 0 makes sense
Mode	most frequent	Categorical columns
Forward-fill	copy previous row	Time-series data

Phase 8 — Visualisation & Dashboard

Chart.js Pattern

```
javascript
```

```

// eda.js - create a chart (always destroy before re-creating)
let _chartInstances = {};

function renderChart(canvasId, config) {
  // Destroy old chart if it exists (prevents memory leak)
  if (_chartInstances[canvasId]) {
    _chartInstances[canvasId].destroy();
  }

  const canvas = document.getElementById(canvasId);
  const ctx = canvas.getContext('2d');
  _chartInstances[canvasId] = new Chart(ctx, config);
}

// Config for a histogram bar chart
function buildChartConfig(ch) {
  const isDark = document.documentElement.getAttribute('data-theme') !== 'light';
  return {
    type: 'bar',
    data: {
      labels: ch.bins, // bin edges from np.histogram()
      datasets: [{ data: ch.counts, backgroundColor: generateColors(ch.bins.length,
    },
    options: {
      responsive: true,
      scales: {
        x: { ticks: { color: isDark ? '#cbd5e1' : '#374151' } },
        y: { ticks: { color: isDark ? '#cbd5e1' : '#374151' } }
      }
    }
  };
}

```

Server-Sent Events (SSE)

python

```
# simulation_engine.py – server pushes events to browser
@simulation_router.get("/simulation/run")
async def run_simulation():
    async def _event_stream():
        for step in SIMULATION_STEPS:
            yield f"data: {json.dumps({'key': step['key'], 'status': 'running'})}\n"
            await asyncio.sleep(step["duration_ms"] / 1000)
            yield f"data: {json.dumps({'key': step['key'], 'status': 'done'})}\n\n"
            yield f"data: {json.dumps({'status': 'complete'})}\n\n"

    return StreamingResponse(_event_stream(), media_type="text/event-stream")
```

javascript

```
// simulation.js – browser listens
const es = new EventSource('/simulation/run');
es.onmessage = (e) => {
    const data = JSON.parse(e.data);
    if (data.status === 'running') _setRunning(data.key);
    if (data.status === 'done') _setDone(data.key, data.section);
    if (data.status === 'complete') es.close();
};
```

Phase 9 — AI & ML Integration

Model Registry Structure

python

```
# ml_engine.py - each entry defines one model
MODEL_REGISTRY = {
    "random_forest_clf": {
        "name": "Random Forest Classifier",
        "category": "Machine Learning",
        "task": ["classification"],
        "complexity": "medium",
        "speed": "medium",
        "icon": "🌲",
        "best_for": ["tabular data", "mixed features", "feature importance"],
        "hyperparams": {"n_estimators": 100, "max_depth": 10, "criterion": "gini"},
        "min_rows": 50,
        "eda_score_min": 50, # requires at least 50% EDA accuracy
    },
    # ... 21 more models
}
```

Full Training Pipeline

```
python
```

```

# ml_engine.py - _preprocess() + train_model()

def _preprocess(df, target_col, task_type):
    y = df[target_col].copy()
    X = df.drop(columns=[target_col])

    # Encode target for classification
    le = None
    if task_type == "classification":
        le = LabelEncoder()
        y = le.fit_transform(y.astype(str))

    # Encode categoricals
    for col in X.select_dtypes(include="object").columns:
        X[col] = LabelEncoder().fit_transform(X[col].astype(str))

    # Impute + scale
    X_arr = SimpleImputer(strategy="median").fit_transform(X.values.astype(float))
    scaler = StandardScaler()
    X_sc = scaler.fit_transform(X_arr)

    return X_sc, y, X.columns.tolist(), scaler, le

def train_model(df, target_col, model_key, hyperparams, test_size=0.2):
    task_type = MODEL_REGISTRY[model_key]["task"][0]
    X, y, feature_names, scaler, le = _preprocess(df, target_col, task_type)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42,
        stratify=y if task_type == "classification" else None
    )

    model = _build_model(model_key, hyperparams)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    return {
        "metrics": _compute_metrics(task_type, y_test, y_pred, model, X_test),
        "feature_importance": _get_feature_importance(model, feature_names, X_train),
        "confusion_matrix": _get_confusion_matrix(y_test, y_pred, le),
        "roc_curve": _get_roc_curve(model, X_test, y_test, le),
        "learning_curve": _get_learning_curve(model, X, y, task_type),
        "model_b64": _encode_object(model),
        "scaler_b64": _encode_object(scaler),
    }

```



```
"le_b64": _encode_object(le) if le else "",  
}
```

Evaluation Metrics Reference

Classification:

- **Accuracy** = correct predictions / total predictions
- **Precision** = $TP / (TP + FP)$ — "of my positive calls, how many were right?"
- **Recall** = $TP / (TP + FN)$ — "of all actual positives, how many did I find?"
- **F1 Score** = $2 \times (Precision \times Recall) / (Precision + Recall)$
- **ROC-AUC** = area under the ROC curve (1.0 = perfect, 0.5 = random)

Regression:

- **MAE** = $\text{mean}(|\text{actual} - \text{predicted}|)$
- **RMSE** = $\sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$
- **R²** = $1 - (SS_{\text{residuals}} / SS_{\text{total}})$ — 1.0 = perfect, 0 = just predicting mean

Model Persistence

python

```

import joblib, base64, io

# Serialise (save model to base64 string for DB storage)
def _encode_object(obj) -> str:
    buf = io.BytesIO()
    joblib.dump(obj, buf)
    return base64.b64encode(buf.getvalue()).decode("utf-8")

# Deserialise (load model from base64 string)
def _decode_object(b64: str):
    return joblib.load(io.BytesIO(base64.b64decode(b64)))

# Predict with saved model
@ml_router.post("/predict")
def ml_predict(body: PredictRequest):
    model = _decode_object(body.model_b64)
    scaler = _decode_object(body.scaler_b64)
    le = _decode_object(body.le_b64) if body.le_b64 else None

    df = pd.DataFrame(body.rows)[body.feature_names]
    X_sc = scaler.transform(df.values.astype(float))
    y_pred = model.predict(X_sc)

    if le:
        predictions = [str(p) for p in le.inverse_transform(y_pred.astype(int))]
    else:
        predictions = [round(float(p), 4) for p in y_pred]

    return {"predictions": predictions}

```

Phase 10 — Startup & Product

Product Thinking Framework

Bad: "Let me add a feature."

Good: "What decision does the user need to make? What's the minimum information they need to make it?"

KoreData's EDA Accuracy score is product thinking in action: instead of showing 20 raw statistics, it computes one number (0-100) with a letter grade and a clear ML-readiness threshold. That is the same information packaged for the decision the user actually needs.

The 3-Layer Priority System

● Critical – broken, data loss, server crash risk

→ Fix before any new features

● High – users can't do core tasks

→ Fix this sprint

● Medium – users want it but work around it

→ Next sprint

● Low – nice to have

→ Backlog

Security Checklist

- ☐ File size limit (100MB) added to `/upload`
- ☐ MIME type validation added
- ☐ Rate limiting (`slowapi`) on upload and auth endpoints
- ☐ All SQL uses `%s` parameters (not f-strings)
- ☐ Secrets in environment variables (not hardcoded)
- ☐ HTTPS enabled on Railway domain
- ☐ OTP expires in 2 minutes
- ☐ Sessions expire in 30 days
- ☐ `bcrypt` rounds = 12 (strong hashing)

Rate Limiting Implementation

```
python

# requirements.txt – add:
# slowapi==0.1.9

# main.py
from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

# On the upload endpoint:
@app.post("/upload")
@limiter.limit("10/hour")
async def upload_file(request: Request, ...):
    ...
```

API Endpoint Reference

Auth Endpoints

Method	Path	Description
POST	/auth/register	Register new user, send OTP
POST	/auth/verify-otp	Verify OTP, create session token
POST	/auth/login-request	OTP login request
POST	/auth/resend-otp	Resend OTP
POST	/auth/password-login	Login with ID + password
POST	/auth/set-password	Set password after OTP verify
POST	/auth/forgot-password/check	Validate login ID exists
POST	/auth/forgot-password/send-otp	Send reset OTP
POST	/auth/forgot-password/verify-otp	Verify reset OTP
POST	/auth/reset-password	Save new password
POST	/auth/logout	Delete session token
GET	/auth/verify	Validate Bearer token

Data Endpoints

Method	Path	Description
POST	/upload	Upload file → run 25-stage EDA
POST	/dataset/edit	Re-run EDA on edited rows
POST	/dataset/select-version	Notify backend of active version
POST	/dataset/apply-missing	Fill one column's missing values
POST	/code-run	Execute Python in sandbox

ML Endpoints

Method	Path	Description
GET	/ml/models	Full model registry
POST	/ml/recommend	EDA-aware model recommendation
POST	/ml/train	Train model + return metrics
POST	/ml/auto	Auto-train top-3 models
POST	/ml/predict	Predict on new rows
POST	/ml/report	Generate JSON report
GET	/ml/saved	List saved models
GET	/ml/history	Training history from DB
GET	/ml/download/{key}	Download .joblib file

Activity Endpoints

Method	Path	Description
POST	/activity/validate	Check saved activity token
POST	/activity/login	Start new activity session
POST	/activity/logout	Close session, record duration
POST	/activity/heartbeat	60s keep-alive
GET	/activity/dashboard	All stat card values
GET	/activity/history	Day-wise history table

Database Schema Reference

kore_users

sql

```
id            INT AUTO_INCREMENT PRIMARY KEY
login_id      VARCHAR(20) UNIQUE NOT NULL          -- e.g. "KD123456"
first_name    VARCHAR(50) NOT NULL
last_name     VARCHAR(50) DEFAULT ''
email         VARCHAR(100) UNIQUE NOT NULL
phone         VARCHAR(20) UNIQUE NOT NULL
is_verified   TINYINT(1) DEFAULT 0
password_hash VARCHAR(255) NULL                    -- NULL = OTP-only account
created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

sessions

```
sql

id            INT AUTO_INCREMENT PRIMARY KEY
token         VARCHAR(64) UNIQUE NOT NULL          -- 64-char hex (secrets.token_hex(32))
login_id      VARCHAR(20) REFERENCES kore_users
created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
expires_at    DATETIME NOT NULL                    -- 30 days from login
```

ml_training_history

```
sql

id            INT AUTO_INCREMENT PRIMARY KEY
login_id      VARCHAR(20) REFERENCES kore_users
model_key     VARCHAR(60)                         -- e.g. "random_forest_clf"
task_type     VARCHAR(20)                         -- classification | regression | other
primary_metric FLOAT                             -- accuracy (clf) | R² (reg) | silhouette
f1_score      FLOAT
rmse          FLOAT
r2_score      FLOAT
cv_score      FLOAT
deploy_ready  TINYINT(1) DEFAULT 0
grade         VARCHAR(20)                         -- Excellent | Good | Fair | Poor
trained_at    DATETIME DEFAULT CURRENT_TIMESTAMP
```

Deployment Guide

Railway Deployment

```
json
```

```
// railway.json
{
  "build": { "builder": "NIXPACKS" },
  "deploy": {
    "startCommand": "uvicorn main:app --host 0.0.0.0 --port $PORT --workers 2",
    "healthcheckPath": "/health",
    "healthcheckTimeout": 300,
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 3
  }
}
```

Environment Variables (set in Railway Dashboard)

```
DB_HOST=<railway mysql host>
DB_PORT=3306
DB_NAME=kore_data
DB_USER=root
DB_PASS=<your password>

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SENDER=your@gmail.com
SMTP_PASSWORD=<gmail app password>

DEMO_MODE=false
OTP_EXPIRY_MINUTES=2
APP_SECRET=<random 32-char string>
```

Startup Sequence

1. Nixpacks detects Python → installs requirements.txt
2. uvicorn starts main:app
3. lifespan() runs: init_db() creates all 11 tables if missing
4. GET /health returns {"status": "ok"} → Railway marks as healthy
5. App serves at your-app.railway.app

Pre-Deploy Checklist

- [] requirements.txt has no git conflict markers
- [] All .py imports are listed in requirements.txt
- [] Environment variables set in Railway dashboard
- [] DB_HOST points to Railway MySQL (not localhost)
- [] app.include_router(landing_router) added to main.py

```
[ ] File size limit added to upload_file()  
[ ] GET /health returns 200  
[ ] No hardcoded passwords or secrets in any .py file
```

Full SQL Schema (kore_complete_schema.sql)

```
sql
```



```

-- 
-- KoreaData v10.0 Complete Schema
-- Run this once on a fresh DB, or re-run safely on existing DB
-- All tables use IF NOT EXISTS – safe to call on every deploy
-- 

-- 1. Core users
CREATE TABLE IF NOT EXISTS kore_users (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    login_id          VARCHAR(20)  NOT NULL UNIQUE,
    first_name        VARCHAR(50)  NOT NULL,
    last_name         VARCHAR(50)  DEFAULT '',
    email             VARCHAR(100) NOT NULL UNIQUE,
    phone             VARCHAR(20)  NOT NULL UNIQUE,
    is_verified       TINYINT(1)   DEFAULT 0,
    password_hash     VARCHAR(255) NULL,
    avatar_url        VARCHAR(255) NULL,
    resume_url        VARCHAR(255) NULL,
    bio               TEXT          NULL,
    location          VARCHAR(100) NULL,
    job_title         VARCHAR(100) NULL,
    created_at        TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at        TIMESTAMP     DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_email    (email),
    INDEX idx_login_id (login_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 2. Auth sessions
CREATE TABLE IF NOT EXISTS sessions (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    token            VARCHAR(64)  NOT NULL UNIQUE,
    login_id         VARCHAR(20)  NOT NULL,
    created_at       DATETIME      DEFAULT CURRENT_TIMESTAMP,
    expires_at       DATETIME      NOT NULL,
    FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
    INDEX idx_token   (token),
    INDEX idx_login_id (login_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. OTP tokens
CREATE TABLE IF NOT EXISTS otp_tokens (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    login_id         VARCHAR(20)  NOT NULL,
    otp_code         VARCHAR(6)   NOT NULL,
    method           VARCHAR(10)  DEFAULT 'email',
    contact          VARCHAR(100) NOT NULL,

```

```
purpose      VARCHAR(20)  DEFAULT 'register',
is_used      TINYINT(1)   DEFAULT 0,
expires_at   DATETIME     NOT NULL,
created_at   DATETIME     DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
INDEX idx_login_otp (login_id, otp_code, is_used)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 4. Uploaded files

```
CREATE TABLE IF NOT EXISTS uploaded_files (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  login_id    VARCHAR(20)  NOT NULL,
  file_name    VARCHAR(255) NOT NULL,
  file_size_kb FLOAT       DEFAULT 0,
  row_count    INT         DEFAULT 0,
  col_count    INT         DEFAULT 0,
  file_type    VARCHAR(10)  DEFAULT 'csv',
  uploaded_at  DATETIME     DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
  INDEX idx_login_uploaded (login_id, uploaded_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 5. Notifications

```
CREATE TABLE IF NOT EXISTS user_notifications (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  login_id    VARCHAR(20)  NOT NULL,
  message     TEXT         NOT NULL,
  type        VARCHAR(20)  DEFAULT 'info',
  is_read     TINYINT(1)   DEFAULT 0,
  link        VARCHAR(255) NULL,
  created_at  DATETIME     DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
  INDEX idx_login_unread (login_id, is_read)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 6. Password reset audit

```
CREATE TABLE IF NOT EXISTS password_reset_log (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  login_id    VARCHAR(20)  NOT NULL,
  action      VARCHAR(50)  NOT NULL,
  ip_address  VARCHAR(45)  NULL,
  created_at  DATETIME     DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_login_action (login_id, action)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 7. User sessions (activity tracking: per login)

```
CREATE TABLE IF NOT EXISTS user_sessions (
```

```

id            INT AUTO_INCREMENT PRIMARY KEY,
login_id      VARCHAR(20) NOT NULL,
session_token VARCHAR(64) NOT NULL UNIQUE,
login_at      DATETIME     DEFAULT CURRENT_TIMESTAMP,
logout_at     DATETIME     NULL,
last_seen_at  DATETIME     DEFAULT CURRENT_TIMESTAMP,
duration_sec  INT          DEFAULT 0,
is_active     TINYINT(1)   DEFAULT 1,
ip_address    VARCHAR(45)  NULL,
user_agent    VARCHAR(255) NULL,
FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
INDEX idx_login_active (login_id, is_active),
INDEX idx_session_token (session_token)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 8. Daily activity (aggregate: one row per user per day)

```

CREATE TABLE IF NOT EXISTS daily_activity (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    login_id      VARCHAR(20) NOT NULL,
    activity_date  DATE        NOT NULL,
    first_login_at DATETIME    NULL,
    last_seen_at  DATETIME    NULL,
    total_active_sec INT       DEFAULT 0,
    login_count   INT         DEFAULT 0,
    logout_count  INT         DEFAULT 0,
    FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
    UNIQUE KEY uq_login_date (login_id, activity_date),
    INDEX idx_date (activity_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 9. User activity log (detailed action trail)

```

CREATE TABLE IF NOT EXISTS user_activity (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    login_id      VARCHAR(20) NOT NULL,
    action        VARCHAR(100) NOT NULL,
    detail        TEXT        NULL,
    ip_address    VARCHAR(45)  NULL,
    device        VARCHAR(100) NULL,
    created_at    DATETIME     DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_login_action (login_id, action),
    INDEX idx_created_at   (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 10. ML training history

```

CREATE TABLE IF NOT EXISTS ml_training_history (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    login_id      VARCHAR(20) NOT NULL,

```

```

model_key      VARCHAR(60)  NOT NULL,
model_name     VARCHAR(100) NOT NULL,
task_type      VARCHAR(20)  NOT NULL,
target_col     VARCHAR(100) NULL,
n_features     INT           DEFAULT 0,
n_rows         INT           DEFAULT 0,
test_size      FLOAT        DEFAULT 0.2,
primary_metric FLOAT        NULL,
accuracy       FLOAT        NULL,
precision_score FLOAT       NULL,
recall_score   FLOAT       NULL,
f1_score       FLOAT       NULL,
roc_auc        FLOAT       NULL,
mae            FLOAT       NULL,
mse            FLOAT       NULL,
rmse           FLOAT       NULL,
r2_score       FLOAT       NULL,
cv_score       FLOAT       NULL,
deploy_ready   TINYINT(1)   DEFAULT 0,
grade          VARCHAR(20)  NULL,
trained_at     DATETIME     DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
INDEX idx_login_trained (login_id, trained_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 11. Saved ML models

```

CREATE TABLE IF NOT EXISTS ml_saved_models (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  login_id    VARCHAR(20)  NOT NULL,
  model_key   VARCHAR(60)  NOT NULL,
  model_name  VARCHAR(100) NOT NULL,
  task_type   VARCHAR(20)  NOT NULL,
  model_data  LONGTEXT     NOT NULL,      -- base64-encoded joblib
  scaler_data LONGTEXT     NULL,
  le_data     LONGTEXT     NULL,
  feature_names TEXT       NULL,         -- JSON array of column names
  target_col  VARCHAR(100) NULL,
  primary_metric FLOAT     NULL,
  is_active   TINYINT(1)   DEFAULT 1,
  saved_at    DATETIME     DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (login_id) REFERENCES kore_users(login_id) ON DELETE CASCADE,
  INDEX idx_login_active (login_id, is_active)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- — Useful Views —————

```

CREATE OR REPLACE VIEW user_stats AS

```

```
SELECT
    u.login_id,
    u.first_name,
    u.email,
    COUNT(DISTINCT f.id) AS total_uploads,
    COUNT(DISTINCT m.id) AS total_models,
    COUNT(DISTINCT s.id) AS total_sessions,
    MAX(s.last_seen_at) AS last_seen
FROM kore_users u
LEFT JOIN uploaded_files f ON f.login_id = u.login_id
LEFT JOIN ml_training_history m ON m.login_id = u.login_id
LEFT JOIN user_sessions s ON s.login_id = u.login_id
GROUP BY u.login_id, u.first_name, u.email;

CREATE OR REPLACE VIEW active_sessions AS
SELECT s.*, u.first_name, u.email
FROM user_sessions s
JOIN kore_users u ON u.login_id = s.login_id
WHERE s.is_active = 1;
```

Working requirements.txt (Clean — No Merge Conflicts)

txt

```
# — Web Framework —————
fastapi==0.111.0
uvicorn[standard]==0.30.1
python-multipart==0.0.9
pydantic==2.7.1

# — Database —————
mysql-connector-python==8.4.0

# — Data Science —————
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.5.0
scipy==1.13.0
openpyxl==3.1.4
pyarrow==16.1.0
xlrd==2.0.1

# — ML (optional boosting libraries) —————
xgboost==2.0.3
lightgbm==4.3.0

# — Model Serialisation —————
joblib==1.4.2

# — Email (OTP sending) —————
# Uses Python stdlib smtplib - no extra package needed

# — Security —————
bcrypt==4.1.3
python-jose[cryptography]==3.3.0

# — Rate Limiting —————
slowapi==0.1.9

# — PDF Export (optional) —————
# reportlab==4.2.0

# — Utilities —————
python-dotenv==1.0.1
```

Complete Bug Fix Code

Fix 1 — Wire `landing_routes.py` into `main.py`

In `main.py`, find the section where you include other routers (near the top of the file, after imports):

```
python

# main.py – add these lines alongside the other include_router calls

from landing_routes import landing_router, init_contact_table

app.include_router(landing_router)

# Also add init_contact_table() inside lifespan:
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    init_contact_table()    # ← ADD THIS LINE
    logger.info("✓ CoreData v9.0 ready")
    yield
```

In `landing_routes.py`, confirm the router and table init look like this:

```
python
```

```

# landing_routes.py
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from database import db_execute, db_fetchone
import logging

logger = logging.getLogger(__name__)
landing_router = APIRouter()

def init_contact_table():
    db_execute("""
        CREATE TABLE IF NOT EXISTS contact_messages (
            id          INT AUTO_INCREMENT PRIMARY KEY,
            name        VARCHAR(100) NOT NULL,
            email       VARCHAR(100) NOT NULL,
            subject     VARCHAR(200) NULL,
            message     TEXT NOT NULL,
            created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
    """, ())

class ContactRequest(BaseModel):
    name: str
    email: str
    subject: str = ""
    message: str

@landing_router.post("/contact")
def submit_contact(body: ContactRequest):
    if not body.name.strip() or not body.message.strip():
        raise HTTPException(400, "Name and message are required")
    db_execute(
        "INSERT INTO contact_messages (name, email, subject, message) "
        "VALUES (%s, %s, %s, %s)",
        (body.name.strip(), body.email.strip(),
         body.subject.strip(), body.message.strip())
    )
    return {"ok": True, "message": "Thank you! We'll be in touch soon."}

```

Fix 2 — File Size + MIME Type Validation

Replace the start of your `upload_file()` function in `main.py`:

```
python
```



```

# main.py – constants near the top of the file (add once)
MAX_FILE_SIZE    = 100 * 1024 * 1024    # 100 MB in bytes
ALLOWED_EXTENSIONS = {".csv", ".xlsx", ".xls", ".json", ".parquet", ".xml"}
ALLOWED_MIME_TYPES = {
    "text/csv",
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    "application/vnd.ms-excel",
    "application/json",
    "text/json",
    "application/octet-stream",    # parquet / generic binary
    "text/xml",
    "application/xml",
}

# Inside upload_file():
@app.post("/upload")
async def upload_file(
    file: UploadFile = File(...),
    authorization: Optional[str] = Header(None)
):
    user = _require_auth(authorization)

    # — Validation —————
    fname = (file.filename or "").lower()
    ext    = "." + fname.rsplit(".", 1)[-1] if "." in fname else ""

    if ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(400, f"File type '{ext}' not supported. "
                                f"Allowed: {'', ' '.join(ALLOWED_EXTENSIONS)}")

    contents = await file.read()

    if len(contents) == 0:
        raise HTTPException(400, "Uploaded file is empty")

    if len(contents) > MAX_FILE_SIZE:
        size_mb = len(contents) / (1024 * 1024)
        raise HTTPException(400, f"File too large ({size_mb:.1f} MB). "
                                f"Maximum allowed size is 100 MB")

    # Optional: MIME type check (not always reliable – filename check above is primary)
    if file.content_type and file.content_type not in ALLOWED_MIME_TYPES:
        logger.warning("Suspicious MIME type '%s' for file '%s'",
                       file.content_type, file.filename)
        # Log but don't block – browsers report MIME inconsistently

```

```

# — Parse —————
try:
    if fname.endswith(".csv"): df = pd.read_csv(io.BytesIO(contents))
    elif fname.endswith((".xlsx", ".xls")): df = pd.read_excel(io.BytesIO(contents))
    elif fname.endswith(".json"): df = pd.read_json(io.BytesIO(contents))
    elif fname.endswith(".parquet"): df = pd.read_parquet(io.BytesIO(contents))
    elif fname.endswith(".xml"): df = pd.read_xml(io.BytesIO(contents))
    else:
        raise HTTPException(400, "Could not determine file format")
except HTTPException:
    raise
except Exception as e:
    raise HTTPException(422, f"Could not parse file: {str(e)}")

# — Continue with EDA —————
eda_result = perform_eda(df)
# ... rest of function unchanged

```

Fix 3 — requirements.txt Merge Conflict (Manual Steps)

1. Open `requirements.txt` in your editor
2. Search for `<<<<<<` — delete the entire conflict block including both versions and all three marker lines
3. The fixed file should contain only clean package lines — use the **Working requirements.txt** section above as reference
4. Run locally to verify: `pip install -r requirements.txt --dry-run`
5. Commit and push: `git add requirements.txt && git commit -m "fix: resolve merge conflict in requirements.txt"`

Rate Limiting Implementation (slowapi)

Full implementation to protect upload, auth, and ML endpoints:

```
python
```

```

# main.py – add at top after imports

from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from fastapi import Request
from fastapi.responses import JSONResponse

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

# Custom error response (returns JSON, not HTML)
@app.exception_handler(RateLimitExceeded)
async def rate_limit_handler(request: Request, exc: RateLimitExceeded):
    return JSONResponse(
        status_code=429,
        content={"detail": f"Rate limit exceeded. Try again later."}
    )

# — Apply to specific endpoints —————

@app.post("/upload")
@limiter.limit("10/hour") # 10 uploads per IP per hour
async def upload_file(request: Request, file: UploadFile = File(...), ...):
    ...

@app.post("/auth/register")
@limiter.limit("5/hour") # 5 register attempts per hour
async def register(request: Request, body: RegisterRequest):
    ...

@app.post("/auth/login-request")
@limiter.limit("10/hour") # 10 OTP login requests per hour
async def login_request(request: Request, body: LoginRequest):
    ...

@app.post("/ml/train")
@limiter.limit("5/hour") # 5 training jobs per hour
async def train_model(request: Request, body: TrainRequest, ...):
    ...

```

Note: The `request: Request` parameter must be the first positional parameter in every rate-limited endpoint — slowapi requires it.

Column & Row Filter Feature (Next Priority Implementation)

Backend

python

```
# main.py – add these two endpoints
```

```
class FilterRequest(BaseModel):
    rows: list[dict]
    columns: list[str]
    # Column filter
    keep_columns: list[str] | None = None # None = keep all
    drop_columns: list[str] | None = None
    # Row filter
    filter_column: str | None = None
    filter_op: str | None = None # "gt", "lt", "eq", "contains", "notnull"
    filter_value: str | None = None

@app.post("/dataset/filter")
async def filter_dataset(
    body: FilterRequest,
    authorization: Optional[str] = Header(None)
):
    user = _require_auth(authorization)
    df = pd.DataFrame(body.rows, columns=body.columns)

    # Column filter
    if body.keep_columns:
        valid = [c for c in body.keep_columns if c in df.columns]
        df = df[valid]
    elif body.drop_columns:
        df = df.drop(columns=[c for c in body.drop_columns if c in df.columns])

    # Row filter
    if body.filter_column and body.filter_column in df.columns:
        col = df[body.filter_column]
        op = body.filter_op or "notnull"
        val = body.filter_value

        if op == "gt": df = df[pd.to_numeric(col, errors='coerce') > float(val)]
        elif op == "lt": df = df[pd.to_numeric(col, errors='coerce') < float(val)]
        elif op == "eq": df = df[col.astype(str) == str(val)]
        elif op == "contains": df = df[col.astype(str).str.contains(str(val), na=False)]
        elif op == "notnull": df = df[col.notna()]

    eda_result = perform_eda(df)
    return {
        "ok": True,
        "filtered_shape": {"rows": len(df), "cols": len(df.columns)},
        "eda": eda_result,
        "rows": clean_json(df.head(500).to_dict(orient="records")),
    }
```

```
    "columns": df.columns.tolist()  
}
```

Frontend (edit_dataset.js additions)

javascript

```
// edit_dataset.js - add to the control panel HTML
// Filter bar appears above the data table

function renderFilterBar(columns) {
  const bar = document.getElementById('filter-bar');
  bar.innerHTML = `
    <div class="filter-row">
      <select id="filter-col">
        <option value="">Filter by column...</option>
        ${columns.map(c => `<option value="${c}">${c}</option>`).join('')}
      </select>
      <select id="filter-op">
        <option value="notnull">Not null</option>
        <option value="eq">Equals</option>
        <option value="contains">Contains</option>
        <option value="gt">Greater than</option>
        <option value="lt">Less than</option>
      </select>
      <input id="filter-val" type="text" placeholder="value...">
      <button onclick="applyFilter()">Apply Filter</button>
      <button onclick="clearFilter()">Clear</button>
    </div>
    <div class="col-toggle">
      ${columns.map(c => `
        <label class="col-chip">
          <input type="checkbox" value="${c}" checked onchange="toggleColumn('${c}',
            ${c}
          </label>
        `).join('')}
    </div>
  `;
}

async function applyFilter() {
  const filterCol = document.getElementById('filter-col').value;
  const filterOp = document.getElementById('filter-op').value;
  const filterVal = document.getElementById('filter-val').value;

  const hiddenCols = [...document.querySelectorAll('.col-chip input:not(:checked)')]
    .map(el => el.value);

  const res = await fetch('/dataset/filter', {
    method: 'POST',
    headers: { ..._authHeaders(), 'Content-Type': 'application/json' },
    body: JSON.stringify({
      rows: _activeRows,

```

```

        columns:      _headers,
        drop_columns: hiddenCols.length ? hiddenCols : null,
        filter_column: filterCol || null,
        filter_op:     filterOp,
        filter_value: filterVal || null
    })
  });
const data = await res.json();
if (data.ok) {
  _renderTable(data.rows, data.columns);
  showToast(`Filtered: ${data.filtered_shape.rows} rows × ${data.filtered_shape.co
}
}
}

```

Health Check Endpoint

Add this to `main.py` — Railway uses it to confirm your app is running:

```

python

from datetime import datetime

@app.get("/health")
def health_check():
    """
    Railway health check — must return 200 to mark deployment as healthy.
    Also checks DB connectivity.
    """
    try:
        db_fetchone("SELECT 1 AS ok", ())
        db_status = "connected"
    except Exception as e:
        db_status = f"error: {str(e)}"

    return {
        "status":      "ok",
        "version":     "9.0",
        "db":          db_status,
        "timestamp":   datetime.utcnow().isoformat() + "Z"
    }

```


Capstone Tasks — All Phases

Phase	Task	File	Difficulty
P2	Add frontend file type + size validation in <code>uploadFile()</code> before the <code>fetch()</code> call	<code>eda.js</code>	★
P3	Add <code>GET /my-stats</code> endpoint returning upload count, model count, last login	<code>main.py</code>	★
P4	Design a <code>user_saved_charts</code> table; write INSERT, SELECT, DELETE queries	SQL	★★
P5	Add a Standard Deviation Ratio insight rule to <code>generate_insights()</code> : if $\text{std} > 2 \times \text{mean}$, flag the column as highly variable	<code>eda_engine.py</code>	★★
P6	Add an email format validity check to <code>compute_eda_accuracy()</code> : scan string columns for email-like values and check if they match <code>@domain.tld</code> pattern	<code>eda_engine.py</code>	★★
P7	Build <code>POST /dataset/rename-column</code> — accepts <code>{old_name, new_name}</code> , renames the column server-side, re-runs EDA, returns fresh result	<code>main.py</code>	★★
P8	Add a line chart for datetime columns: detect the first datetime column, plot value over time, render in the Charts section	<code>eda.js</code>	★★★
P9	Add class imbalance factor to <code>recommend_models()</code> : if minority class $< 10\%$ of rows, penalise Logistic Regression, boost RF + SVM	<code>ml_engine.py</code>	★★★
P10	Full production-readiness audit: go through every item in the Pre-Deploy Checklist and the Security Checklist	All files	★★★

Glossary

Term	Definition
API endpoint	A URL that receives a specific type of HTTP request and returns data
Bearer token	A string (64-char hex) sent in the <code>Authorization</code> header to prove identity

Term	Definition
bcrypt	A slow password hashing algorithm — slower = harder to brute force
DataFrame	pandas' core data structure — a 2D table of rows and columns
EDA	Exploratory Data Analysis — automated statistical summary of a dataset
Feature	A column used as input to an ML model
Idempotent	Safe to run multiple times — produces the same result. <code>CREATE TABLE IF NOT EXISTS</code> is idempotent.
IQR	Interquartile Range: $Q3 - Q1$. Outlier fence = $Q1 - 1.5 \times IQR$ to $Q3 + 1.5 \times IQR$
joblib	Python library for serialising ML models to binary files
OTP	One-Time Password — a 6-digit code that expires after use or 2 minutes
Pydantic	Python library for data validation — FastAPI uses it for request/response models
Recall	$TP / (TP + FN)$ — "of all actual positive cases, how many did the model find?"
ROC-AUC	Area Under the ROC Curve — classifier quality metric. 0.5 = random, 1.0 = perfect
R²	Coefficient of determination — how much variance the model explains (1.0 = perfect)
Skewness	Asymmetry of a distribution. 0 = symmetric; > 0 = right-tailed; < 0 = left-tailed
SSE	Server-Sent Events — one-directional stream from server to browser
Stratify	Keep the same class distribution in both train/test splits
Train/test split	Divide data: 80% to train the model, 20% to evaluate it on unseen data
Upsert	<code>INSERT ... ON DUPLICATE KEY UPDATE</code> — insert a row or update it if it already exists

KoreData v9.0 · FastAPI + MySQL + Vanilla JS + scikit-learn · Built on Railway